

HelixVPN — Refined Architecture & Engineering Specification

HelixVPN — Refined Architecture & Engineering Specification

Revision: 2 Last modified: 2026-07-04T12:00:00Z Status: Refining pass over Home_VPN.md — **primary architectural source, now elaborated by docs/research/mvp/final/** **Provenance note (added Rev 2, 2026-07-04 hardening pass):** This document ([04_ARCH]) is the dominant architectural source cited throughout docs/research/mvp/final/ — SPECIFICATION.md §12 names it the primary source, and 99-source-coverage-ledger.md records it as the most-cited source in the corpus. Since this document was written, docs/research/mvp/final/ has grown into a ~140-nano-detail-doc specification (SPECIFICATION.md is now the architectural **spine**; v00-meta/decision-register.md is now the canonical decision register expanding §13 below) that elaborates every section of this document to phase→task→subtask and near-code granularity, and which supersedes this document in point-of-fact detail (exact numeric SLOs, DDL, protobuf, per- platform mechanics). **Where this document and final/ disagree, final/ (specifically SPECIFICATION.md and v00-meta/decision-register.md) wins** — this document remains the authoritative **readable narrative overview** of the architecture and is not being rewritten section-by-section to chase every downstream refinement; it was independently re-checked during the 2026-07-04 hardening pass for any *outright* contradiction with final/ (none found — every open decision/risk in §13 below is still genuinely open or has since been resolved consistently with what §13 anticipated, e.g. D-PKI-CA-TIER and D-OD-1 are now operator-confirmed per decision-register.md, and the Rust-core/Rust-edge preference in §13 still awaits gates G3/G4 as this document already said it would). **Scope of this document:** Turn the original deployment guide (a sysadmin-grade Hysteria2/WireGuard "how to set up a VPS tunnel" walkthrough) into a real product/engineering spec for a

self-hostable, two-way, multi-network VPN gateway platform with Mullvad-parity privacy features and a cross-platform app suite on a shared codebase.

0. What this refinement changes (gap analysis vs. the original research)

The original Home_VPN.md is solid as an *ops runbook* for a single-purpose reverse tunnel. But measured against the actual brief, it has structural gaps. This spec closes them.

Brief requirement	Original doc	This spec
VPS is a gateway to multiple joined networks , each exposed to end users	Single home LAN behind one tunnel	First-class multi-connector overlay : N networks advertise prefixes, gateway stitches + polices them
Two-way VPN (network-side tunnels <i>in</i> , users tunnel <i>in</i> , traffic routed <i>between</i>)	One-way "reverse tunnel for my home"	Explicit Connector $\rightleftharpoons$ Gateway $\rightleftharpoons$ Client three-role model with a control plane
Apps on both sides (end-user access + network-side connector) + admin/config apps	Third-party clients only (Hiddify, Shadowrocket, WireGuard app)	Three first-party app classes on one shared codebase
All Mullvad power features , esp. QUIC obfuscation	Hysteria2 (a <i>different</i> protocol) suggested as the QUIC path	WireGuard core + MASQUE (RFC 9298) QUIC obfuscation — the <i>actual</i> Mullvad mechanism — plus the full obfuscation stack
Stack: Go / Gin / Postgres / Redis / Podman	Bash + systemd + Docker only	Go/Gin control plane, Postgres+RLS, Redis Streams, Podman quadlets (rootless)
Cross-platform apps (iOS, Android, Aurora, HarmonyOS, Win, Linux, macOS, Web) with max code	None	Rust data-plane core + Flutter UI , the only combo reaching all 8 targets

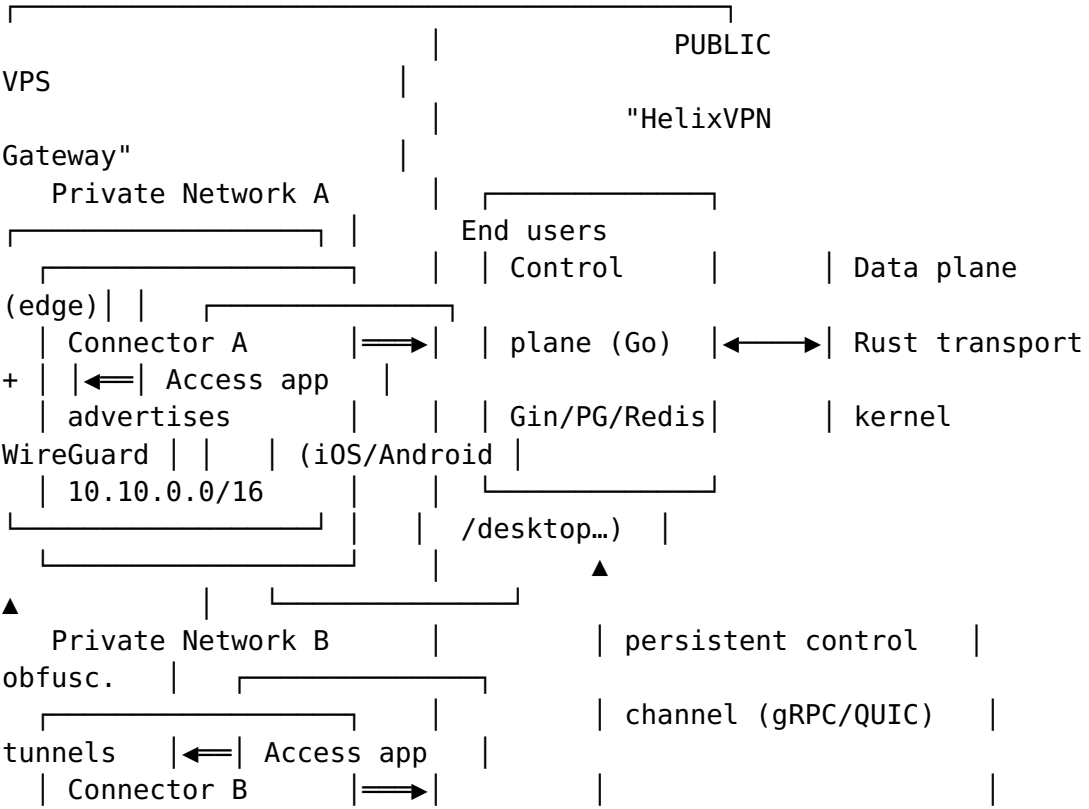
Brief requirement	Original doc	This spec
reuse, tiny size, fast		Event bus + persistent control channels + push-based "network map" reconciliation
Event-driven, real-time end to end	Polling / cron / manual restarts	

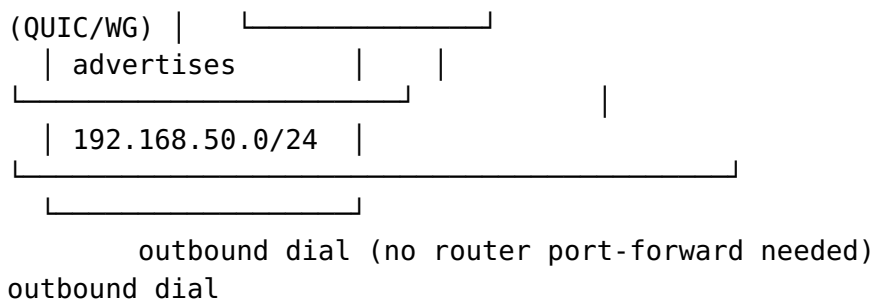
The single most important correction: *Mullvad's QUIC mode is not a separate protocol — it is WireGuard tunneled over MASQUE/HTTP-3.* So the design keeps **WireGuard as the cryptographic core** and treats QUIC/Hysteria/Shadowsocks/UDP-over-TCP as **interchangeable obfuscating transports** underneath it. That decision drives everything else.

1. Product definition

HelixVPN is a **self-hosted overlay network with a privacy-VPN front end**. One sentence: *it is Cloudflare Tunnel + WARP, rebuilt as Tailscale-style coordination, with Mullvad's obfuscation stack, fully self-hostable, on one Rust+Flutter app codebase.*

1.1 The three roles





- **Connector** (network-side): a first-party agent installed on a host *inside* each private network. It **dials outbound** to the gateway (no inbound port-forwarding — this preserves the original doc's core insight), authenticates, and **advertises the prefixes** that network exposes. It is the in-network half of the "two-way" connection.
- **Gateway** (the VPS): the rendezvous hub. It runs a **control plane** (Go) and a **data plane** (Rust edge + kernel WireGuard). It authenticates connectors and clients, maintains the routing/policy table, and **relays/routes** traffic between users and the networks they're authorized to reach.
- **Client** (end-user): the access app, Mullvad-style. Dials into the gateway, receives an overlay IP, and reaches the subset of joined networks its policy allows. Can also use the gateway as a plain privacy exit (full-tunnel to internet).

"Multiple joined networks exposed through it" = multiple connectors, each advertising prefixes, stitched into one policed overlay. "Two-way VPN" = both halves (connector and client) initiate **outbound** tunnels to the gateway, which routes between them.

1.2 Personas / app classes

App class	User	Primary jobs	Platforms
Helix Access	End user	Connect, pick exit/network, toggle obfuscation, kill-switch, split-tunnel	iOS, Android, Aurora, HarmonyOS, Windows, Linux, macOS (Web = limited, see §5.7)
Helix Connector	Network operator	Onboard a network, advertise CIDRs, set local ACLs, run headless	Linux/Windows/macOS (daemon + optional Flutter UI); Android/embedded for appliance use
Helix Console	Admin	Tenants, users, devices, networks, routes,	Web (responsive) + Desktop (same Flutter build)

App class	User	Primary jobs	Platforms
		policies, audit, billing-optional	

All three share **one design system, one Dart UI core, one API client**. Access + Connector additionally share **one Rust VPN/transport core**. (See §5.)

1.3 Prior art and where HelixVPN sits

System	Self-host	Obfuscation	Multi-network overlay	Cross-platform first-party apps	Notes
Mullvad	✗ (service)	best-in-class (QUIC/MASQUE, Shadowsocks, UoT, LWO, DAITA)	✗ (privacy exit only)	✗	The obfuscation privacy bar to m
Tailscale	✗ partial (Headscale)	✗	✗ (mesh + subnet routers)	✗	The coordination/"n map" model to k
NetBird		limited	✗	✗	Closest OSS sha WireGuard + m signal server
Cloudflare Tunnel + WARP	✗	✗ MASQUE	✗	✗	The "connector out" + MASQUE model
Twingate / Zscaler	✗	n/a		ZTNA	ZTNA policy mo
HelixVPN		full Mullvad-parity stack		on 8 platforms, one codebase	Self-hosted unio the above

Differentiators to lead with: (1) self-hosted *and* Mullvad-grade obfuscation including MASQUE/QUIC; (2) genuine 8-platform reach including Aurora + HarmonyOS that no incumbent ships; (3) one shared Rust+Flutter codebase; (4) event-driven real-time control plane.

2. Architectural principles (non-negotiable)

1. **Control plane and data plane are strictly separated.** The Go services never sit in the packet path. If the control plane is down, existing tunnels keep forwarding (fail-static).
 2. **WireGuard is the cryptographic core; transports are pluggable.** Obfuscation is a swappable layer *under* WireGuard, never a fork of the crypto.
 3. **Outbound-only from edges.** Connectors and clients always dial the gateway. No private network ever needs an inbound hole. (Carried forward from the original doc — it's the correct foundation.)
 4. **Push, don't poll.** State changes propagate over persistent channels as events; agents reconcile to a declared desired-state ("network map"), like Tailscale's MapResponse. Cron-based restart loops from the original doc are replaced by event-driven reconciliation + supervised health.
 5. **One core per concern, reused everywhere.** Rust transport/VPN core shared by client, connector, *and* gateway edge. Go domain libraries shared across control services. Dart UI core shared across all three apps.
 6. **Self-hostable by one person, scalable to many gateways.** Single-node podman deploy for a homelab; the same images scale to an HA, multi-region fleet.
 7. **No-logging by construction.** Data plane keeps only counters and ephemeral routing state; no connection/content logs. Privacy is a build property, not a config toggle.
-

3. Data plane — tunnels, transports, obfuscation

3.1 Layering model

L4	Application traffic (user's packets / advertised LAN)	
L3	WireGuard (Noise IK, ChaCha20-Poly1305) ← CRYPTO CORE	
L2	Pluggable transport / obfuscation (one of): • plain UDP (fast path) • MASQUE CONNECT-UDP over HTTP/3 ← the QUIC mode • Shadowsocks-wrap • UDP-over-TCP • LW0 (lightweight WG obfuscation) • port-hopping / 443 / 53 disguise	

| L1 IP to the gateway's public endpoint |

(optional, orthogonal) DAITA-style traffic shaping: constant packet

sizing + cover traffic, applied above WireGuard.

WireGuard handles confidentiality, integrity, and roaming. The **transport layer only changes how the encrypted WG datagrams look on the wire** so DPI/censors can't fingerprint or block them. This is exactly Mullvad's model, and it's why QUIC support is "wrap WG in MASQUE," not "replace WG with Hysteria."

3.2 Transport matrix

Transport	Looks like	Use when	Library (client+edge, Rust)	Cost
Plain WG/UDP	WireGuard	Default; unrestricted networks	kernel WG / boringtun	lowest latency, lowest CPU
MASQUE CONNECT-UDP / HTTP-3	HTTPS / web traffic	Censorship, UDP-throttling, DPI; the QUIC requirement	quinn + h3 + masque layer	moderate; QUIC overhead
MASQUE CONNECT-IP (RFC 9484)	HTTPS	Native IP-over-HTTP/3 datapath (advanced, no inner WG)	quinn + h3	moderate
Shadowsocks-wrap	random/TLS-ish TCP	QUIC blocked, China-style DPI	shadowsocks-rust core	moderate
UDP-over-TCP	TCP	UDP fully blocked	in-house	higher latency
LWO (lightweight obfs)	mangled UDP	Cheap evasion of naive WG signature blocks	in-house (XOR/padding scheme)	near-zero
Port hopping / 443 / 53	varies	Port-based blocks	edge listener config	none

Selection is **automatic with manual override** (Mullvad's exact UX): the client tries plain WG, and after N failed handshakes escalates to LWO → QUIC/MASQUE → Shadowsocks → UoT. The user can pin any mode.

Implementation reuse win: all of these live in **one Rust crate, helix-transport**, consumed identically by the client core, the connector, and the gateway edge. The QUIC/MASQUE code that obfuscates on the client is the *same code* that de-obfuscates on the gateway. One implementation, three consumers.

3.3 QUIC / MASQUE mode in detail (the headline feature)

- Client wraps each WireGuard UDP datagram in an HTTP/3 **CONNECT-UDP** stream (RFC 9298) to the gateway's :443/udp HTTP/3 listener.
- To any observer (and to most DPI) the flow is indistinguishable from a browser talking HTTP/3 to a web server. The gateway edge can **masquerade** unmatched/probe traffic as a real website (carry forward the original doc's Nginx-camouflage idea, but native in the edge).
- The gateway edge terminates QUIC, unwraps the WG datagrams, and hands them to **kernel WireGuard** on the fast path.
- Inherits QUIC's loss recovery + multiplexing — better than UDP-over-TCP on lossy mobile networks, which is precisely why Mullvad shipped it for mobile.

Go is acceptable here (quic-go + masque-go are mature), but **Rust (quinn) is chosen for the edge** so the obfuscation logic is shared byte-for-byte with the clients and to keep the hot path off the GC.

3.4 Multi-network routing and policy

This is the part the original doc never addressed.

- **Overlay addressing:** every node (client, connector, advertised host) gets a stable overlay address. Use a ULA IPv6 /48 per tenant (e.g., fd7a:helix:<tenant>::/48); map advertised IPv4 LANs via a Tailscale-style **4via6** scheme so overlapping 192.168.1.0/24s across different connectors never collide.
- **Prefix advertisement:** connectors advertise their served CIDRs to the control plane; the control plane compiles a **routing map** (which connector is the next hop for which prefix) and pushes it to the gateway edge and to authorized clients.
- **Policy/ACL engine:** a declarative allow-list (group:contractors → net:warehouse-cameras:554/tcp) evaluated at the gateway.

Default-deny. Compiled to per-peer AllowedIPs + an nftables/eBPF verdict map on the edge.

- **Overlapping CIDR handling:** when two connectors expose the same RFC1918 range, the gateway presents each as a distinct overlay prefix and NATs into the connector. Documented as a first-class scenario, not an afterthought.
- **Split horizon / segmentation:** connectors can't reach each other unless policy says so; clients can't reach networks they're not granted. Microsegmentation is the default.

3.5 Multi-hop

Generalize the original doc's "chain two VPSes." A client may route Client → Gateway-Entry → Gateway-Exit → {internet | connector}. Entry sees the client but not the destination; exit sees the destination but not the client. Implemented as nested WireGuard with per-hop keys (the Mullvad multi-hop model), orchestrated by the control plane and pushed as a multi-hop network map.

4. Control plane (Go / Gin / Postgres / Redis / Podman)

The control plane never touches packets. It is the source of truth for identity, devices, networks, routes, and policy, and it distributes desired-state to the edges in real time.

4.1 Services (modular monolith first, split later)

Service	Responsibility	Tech
identity	Tenants, users, SSO/OIDC, device enrollment, API tokens	Go, Gin, Postgres (RLS)
registry	Devices, connectors, advertised prefixes, overlay IP allocation	Go, Postgres, Redis (ephemeral presence)
policy	ACL model, compilation to per-peer rule sets	Go (CUE/Rego-style evaluation)
coordinator	The brain: builds network maps , streams desired-state to agents, reconciles	Go, gRPC server-streaming, Redis Streams
pki	WireGuard key registry, short-lived device certs, rotation, PQ handshake material	Go, Postgres, optional KMS/HSM
telemetry		

Service	Responsibility	Tech
	Counters, health, audit events (no traffic logs)	Go, Prometheus exposition, ClickHouse optional
api-gateway	REST (apps) + gRPC (agents) + WS/SSE (live UI) fan-in	Gin + Connect-Go

Start as **one Go binary, many packages** (the original doc's single-VPS reality), deployable as one podman container. The service boundaries above are package boundaries from day one so they can be split into separate pods when a deployment grows to a fleet.

4.2 API surface

- **Agents (connectors, client cores): gRPC over Connect** (HTTP/2, or HTTP/3 to share the QUIC stack). The key call is a **server-streaming WatchNetworkMap** — the agent opens it once and receives a snapshot then a delta stream. This replaces all polling.
- **Apps (Access/Connector/Console UI): REST via Gin** for CRUD + **WebSocket/SSE** for live updates (device came online, route changed, handshake failing).
- **Schema-first:** Protobuf for agent contracts, OpenAPI for REST, generated clients for Dart/Go/Rust so the three codebases never drift.

4.3 Event-driven backbone

- **Baseline: Redis Streams** (honoring the requested stack). One stream per concern (events:devices, events:routes, events:policy, events:presence), consumer groups per service, XADD/XREADGROUP, dead-letter via pending-entries list.
- **Scale option: NATS JetStream** when you outgrow a single Redis (multi-region fan-out, durable subjects). The event taxonomy is bus-agnostic so the swap is a transport change, not a redesign.
- **Event taxonomy (examples):**
 - device.enrolled, device.online, device.offline, device.revoked
 - connector.attached, connector.prefixes.changed, connector.health.degraded
 - route.advertised, route.withdrawn, route.conflict.detected
 - policy.updated, policy.compiled
 - gateway.failover, gateway.capacity.warning
- Every state-changing API write **emits an event**; the coordinator consumes events, recomputes affected network maps, and pushes deltas. This is the spine that makes "everything real-time" true.

4.4 Real-time state distribution ("network map" model)

Borrowed from Tailscale, adapted:

1. Agent connects, authenticates, opens WatchNetworkMap.
2. Coordinator sends a **full snapshot** (this agent's overlay IP, peers it may reach, routes, transport policy, DNS, kill-switch posture).
3. On any relevant event, coordinator computes the **minimal delta** for each affected agent and pushes it on the open stream.
4. Agents are **declarative reconcilers**: they diff desired-vs-actual and converge (bring up/down peers, change transport, update routes) with zero polling and no restarts.

Convergence target: a policy or route change is reflected on all affected edges in **< 1 second** under normal load.

4.5 Data model (Postgres, with Row-Level Security per tenant)

Sketch (not exhaustive):

```
-- every table carries tenant_id and is guarded by RLS
tenants(id, name, created_at)
users(id, tenant_id, email, oidc_sub, role)
    -- role: admin|operator|member
devices(id, tenant_id, user_id, kind, pubkey, overlay_ip,
    -- kind: client|connector
    last_seen, enrolled_at, revoked_at)
connectors(device_id, tenant_id, site_name)
advertised_prefixes(id, connector_id, cidr, via, enabled)
overlay_allocations(tenant_id, cidr, next_ip)          -- IP
    allocator state
policies(id, tenant_id, spec_jsonb, version, compiled_at)
policy_rules(policy_id, src_group, dst_selector, ports, action)
groups(id, tenant_id, name)  group_members(group_id, device_id)
audit_events(id, tenant_id, actor, action, target, ts, meta_jsonb)
```

- **RLS** enforces tenant isolation at the database, not just the app layer — defense in depth and a clean multi-tenant story for the self-hoster who runs networks for multiple clients.
- **No connections or traffic table.** Live session/presence state lives in **Redis** (ephemeral, TTL'd) so the durable store never accumulates a connection log. This operationalizes the no-logging promise.

4.6 Redis usage

Sessions/presence (device:<id>:presence, TTL refreshed by heartbeat), the **ephemeral routing table** the edge reads, rate limiting (token buckets per API key), pub/sub for low-latency intra-node signaling, and the event Streams above.

4.7 Deployment substrate — Podman (rootless quadlets)

- Ship every component as an OCI image; run with **Podman quadlets** (.container units managed by systemd) — rootless by default, no Docker daemon, better for a security product.
 - A **pod** groups gateway-edge + control plane + Postgres + Redis for single-node self-host; multi-region splits them.
 - Provide a `helixvpctl` CLI (Go, Cobra) that generates quadlets, keys, and the first admin — the modern replacement for the original doc's pile of bash install scripts.
-

5. Client / app architecture — the shared-codebase strategy

This is the heart of the brief: *all apps, all platforms, maximum reuse, tiny, fast*. The answer is **two shared cores with thin per-platform shims**, not one framework doing everything.

5.1 Why two cores

A VPN client is two very different programs fused together:

1. A **data-plane core** that must do crypto, packet I/O, and obfuscation at line rate with bounded memory. This **cannot** be Dart/Flutter and **must not** be reimplemented per platform.
2. A **UI/orchestration layer** (screens, settings, account, network picker) that benefits enormously from a single cross-platform toolkit.

So:

- **helix-core (Rust)**: WireGuard handling (boringtun userspace or kernel control), the helix-transport obfuscation crate (§3.2), kill-switch logic, DNS handling, network-map reconciliation, FFI surface. Compiled to a static lib / cdylib per platform. **This is the same precedent as Mullvad (Rust daemon) and Cloudflare (Rust connector/WARP)**. Exposed to Dart via `flutter_rust_bridge`; exposed to native shims via UniFFI where needed.

- **helix-ui (Flutter/Dart):** every screen, the design system, the API/WebSocket client, state management. One codebase for all three app classes via flavors.

5.2 Why Flutter (and not Compose Multiplatform)

The brief demands **iOS, Android, Aurora, HarmonyOS, Windows, Linux, macOS, Web**. Only Flutter reaches all of them:

Target	Flutter path	KMP / Compose MP
iOS / Android	mainline	
Windows / Linux / macOS	mainline	(Desktop JVM)
Web	mainline (CanvasKit/Wasm)	(Wasm/JS)
Aurora OS	OMP Russia fork gitlab.com/omprussia/flutter (flutter-aurora, builds signed RPM); plugins by Friflex	✗ none
HarmonyOS NEXT	OpenHarmony SIG fork gitee.com/openharmony-sig/flutter_flutter (ohos channel, builds HAP); platform plugins in ArkTS via MethodChannel	none

HarmonyOS NEXT dropped Android/ART compatibility entirely (native ArkTS/ArkUI/DevEco only), and Aurora is Qt/QML-native — so an Android APK or a KMP artifact simply will not run on either. The Flutter forks are the only realistic single-codebase path, and even the Aurora SDK comparison literature concedes Flutter as the cross-platform route. **Conclusion: Flutter for UI; KMP is not viable for the full target set.**

5.3 Per-platform tunnel shims

The Rust core needs an OS-level TUN/packet path, which is always platform-native. Thin shims (a few hundred lines each) own only that:

Platform	Tunnel mechanism	Shim language
iOS / macOS	NEPacketTunnelProvider (Network Extension)	Swift
Android	VpnService + JNI to helix-core	Kotlin

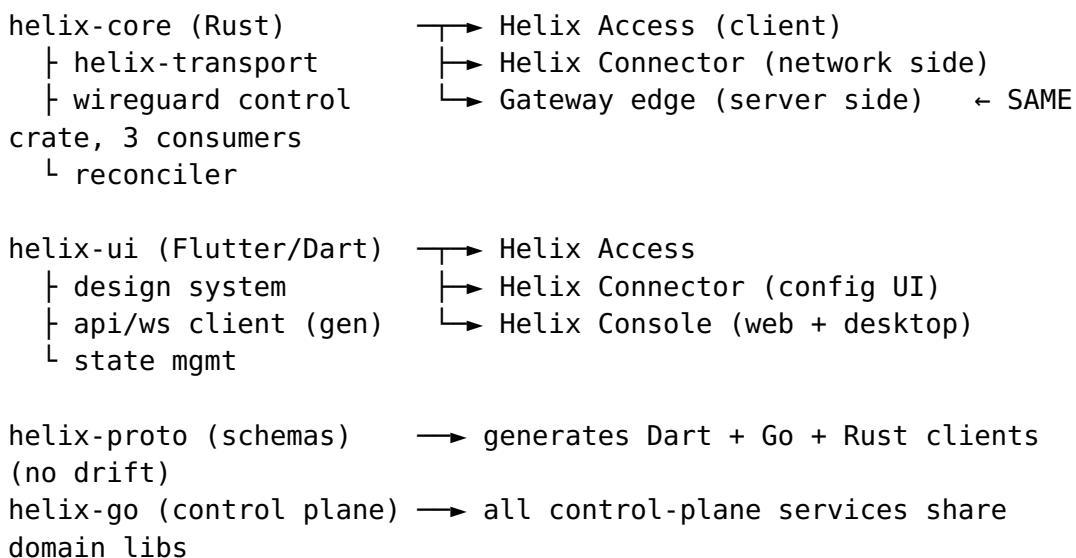
Platform	Tunnel mechanism	Shim language
Windows	wireguard-nt / wintun	Rust + small C#/Win service
Linux	kernel wireguard or tun	Rust
HarmonyOS NEXT	Network Kit VPN extension ability	ArkTS shim → NAPI → helix-core
Aurora OS	Qt/C++ network backend + tun	C++ shim → helix-core
Web	no OS tunnel possible — see §5.7	Dart + WASM

Everything above the shim (protocol, obfuscation, reconciliation, UI) is shared. Per-platform code is the irreducible minimum.

5.4 The three app classes from one tree

- **Helix Access** = helix-ui (full) + helix-core + tunnel shim. The Mullvad-style consumer app.
- **Helix Connector** = helix-core (advertise/route mode) + a **headless daemon** entrypoint + an *optional* slim helix-ui config surface. Same Rust core, different run mode (it advertises prefixes and routes LAN traffic rather than capturing the device's traffic).
- **Helix Console** = helix-ui (admin flavor) + API client only (no helix-core, no tunnel). Builds to Web (responsive) and Desktop from the identical Flutter project.

5.5 Reuse map



Three reuse pillars: **Rust core** (data plane, client+connector+edge), **Flutter UI** (all apps), **schema-generated clients** (all languages).

5.6 Size, memory, speed strategy

- **No Electron, ever.** Desktop apps are Flutter AOT — tens of MB, not hundreds.
- Rust core compiled --release with LTO + strip; small static lib, no runtime/GC, bounded memory — critical for iOS Network Extension memory limits (historically ~15 MB working set) where a Go data plane would be risky.
- Per-ABI split APK/AAB; tree-shaken Flutter; deferred-loaded admin screens.
- Kernel WireGuard on the fast path wherever available; boringtun userspace only as fallback.
- Target: cold start < 1 s on mid-range mobile, idle RSS for the tunnel core in single-digit MB.

5.7 Honest platform caveats (do not hand-wave these)

- **Web cannot run a real device VPN.** Browsers can't open a TUN device. The web build is therefore **Helix Console (management) + account/config**, plus an *optional* in-page **WASM MASQUE client** that can proxy *the browser's own* traffic to a joined network — not a system-wide tunnel. State this plainly to users; "fully responsive web app" = the console and a browser-scoped proxy, not a full VPN.
- **Aurora toolchain is Russian-hosted** (gitlab.com/omprussia, Mos.Hub) and Aurora is primarily a government/enterprise platform. Plan CI runners and signing accordingly; treat it as an enterprise SKU, not a consumer afterthought.
- **HarmonyOS Flutter fork lags mainline** Flutter versions and needs DevEco signing + ArkTS plugin work for the VPN extension. Budget real platform-specific effort for the tunnel ability; the *UI* ports cheaply, the *tunnel shim* does not.
- **iOS Network Extension memory ceiling** is the single hardest constraint and the strongest reason the core is Rust.

6. Mullvad feature-parity matrix → HelixVPN implementation

Mullvad feature	What it does	HelixVPN implementation
WireGuard-only crypto	Modern, audited tunnel	

Mullvad feature	What it does	HelixVPN implementation
		helix-core WireGuard (kernel fast path, boringtun fallback)
QUIC obfuscation (MASQUE/ RFC 9298)	WG-over-HTTP/3, looks like web	helix-transport CONNECT-UDP via quinn+h3; edge terminates on :443/udp
Shadowsocks obfuscation	WG-in-Shadowsocks	helix-transport Shadowsocks wrap
UDP-over-TCP	WG when UDP blocked	helix-transport UoT
LWO (lightweight obfs)	cheap WG signature evasion	helix-transport LWO scheme
Automatic obfuscation	try methods until one works	client escalation ladder (§3.2), driven by handshake failure events
Custom WG port / 443 / 53	port-based evasion	edge multi-listener + port-hopping
DAITA (anti traffic-analysis)	constant packet size + cover traffic	optional shaping layer above WG (maybe-not-style state machine)
Multi-hop	entry/exit separation	nested WG, control-plane orchestrated (§3.5)
Kill-switch	no leaks if tunnel drops	OS firewall rules driven by helix-core state machine (carry forward the doc's nftables/fwmark rules, but managed by the core, not hand-edited)
Split tunneling	per-app/route bypass	per-route AllowedIPs + per-app rules on Android/desktop
DNS leak protection	force tunnel DNS	core sets tunnel DNS; blocks plaintext :53 off-tunnel
No-logging	no connection logs	architectural: ephemeral Redis presence, no durable connection table (§4.5)
Account # (no email)	anonymous identity	optional anonymous device-token enrollment alongside OIDC
Post-quantum handshake	PQ-safe key exchange	WG PQ pre-shared layer / Kyber-style KEM in pki + core

Mullvad feature	What it does	HelixVPN implementation
Per-device management	see/revoke devices	registry + Console; revoke emits <code>device.revoked</code> → instant edge enforcement

This table is the acceptance checklist for "all Mullvad power features." Everything maps to a concrete component; nothing is aspirational.

7. Security architecture

- **Zero-trust, default-deny.** No peer reaches anything without an explicit compiled policy rule.
- **Identity:** OIDC SSO for managed tenants; anonymous device tokens for privacy users. Every device has its own keypair and a short-lived enrollment cert from pki.
- **Transport auth:** mutual auth on the agent control channel (device cert + token); WireGuard's own Noise handshake on the data channel.
- **Key hierarchy:** per-device WG keys, rotated on schedule and on `device.revoked`; gateway keys rotated independently; optional PQ pre-shared layer.
- **Edge hardening:** rootless Podman, read-only rootfs, seccomp, `NET_ADMIN` only where required, no SSH on the data-plane container (manage via control plane).
- **No-logging as code:** the only persistent traffic-derived data is aggregate counters for metrics; assert this with a schema lint in CI (fail the build if a `connection/traffic/packet` durable table appears).
- **Audit:** all *control* actions (not traffic) are audited to `audit_events` and streamed to the Console live.
- **Carry-forward from original doc, upgraded:** fail2ban/SSH-key hardening → replaced by no-public-SSH + WireGuard-only management; GPG/KMS encrypted backups (\$10) retained; the kill-switch iptables rules retained but owned by the core.

8. Event-driven & real-time flows (sequence sketches)

Connector onboarding

```

Connector --(enroll token)--> api: POST /connectors
api --> pki: issue cert + overlay IP          api --> registry:
create device
registry --emit--> connector.attached        coordinator: build map

```

```
for tenant
Connector --open--> WatchNetworkMap (gRPC stream)
coordinator --> Connector: snapshot (peers, routes, transport
policy)
Connector: advertise 10.10.0.0/16 --> registry --emit-->
route.advertised
coordinator: recompute --> push deltas to gateway edge +
authorized clients (<1s)
```

Client connect + reach a joined network

```
Client app --> core: connect                core --> edge: WG
handshake (transport=auto)
  (auto ladder: plain UDP → LWO → QUIC/MASQUE → Shadowsocks → UoT)
core --open--> WatchNetworkMap              coordinator --> core: map
(allowed prefixes)
User opens 10.10.5.20 (camera on Network A)
edge: policy verdict ALLOW --> route via Connector A --> packet
delivered
edge: counters++ (no log)                   Console: live "client
online" via WS
```

Policy change propagation (the real-time proof)

```
Admin (Console) revokes group:contractors access to net:warehouse
api: write policy --emit--> policy.updated
policy svc: compile --emit--> policy.compiled
coordinator: diff affected agents --> push deltas
edge + affected clients: reconcile verdict maps    elapsed: < 1s,
no restarts
```

Gateway failover

```
health probe fails --emit--> gateway.failover
coordinator: reassign clients to standby gateway (floating IP /
Anycast)
clients: map delta → re-handshake to new endpoint, transport
preserved
```

9. Observability (reframed from the original doc)

Keep the original's Prometheus + Grafana + Alertmanager + Blackbox stack — it's good — but:

- Scrape **per-component** metrics: control-plane (Go runtime, event lag, map-push latency), edge (handshakes, transport-mode distribution, QUIC retransmits, bytes per peer — **counts, never content**), connectors (advertised-prefix health, RTT).

- Add **event-lag** and **map-convergence-time** as first-class SLOs (the real-time promise must be measurable: alert if convergence p99 > 1 s).
 - Grafana dashboards: Fleet Overview, Per-Tenant, Transport/Obfuscation Mix, Censorship-Evasion Success Rate (how often the auto-ladder had to escalate, by region).
 - Ship dashboards as code; deploy the whole stack as Podman quadlets alongside the platform (reuse the original doc's compose, ported to quadlets).
-

10. Deployment, HA, and DR

- **Single-node self-host:** one podman pod (edge + control + Postgres + Redis), `helixvpncctl init` bootstraps keys + admin. This is the homelab/original-doc use case, modernized.
 - **Fleet:** multiple gateways (multi-region), shared control plane (or regional control planes federating over NATS), Postgres primary + replicas, Redis/NATS for cross-region events. Clients reach the nearest gateway via Anycast or GeoDNS; floating IPs for fast failover (retain the original doc's DigitalOcean floating-IP pattern).
 - **IaC:** keep the original Terraform for gateway provisioning; cloud-init installs Podman + pulls images + joins the control plane (replace the bash WG/Hysteria install with a single `helixvpncctl join`).
 - **GitOps:** the original doc's sops/git-crypt + PR-driven config model maps cleanly onto policy-as-code here — policies and network definitions live in Git, CI validates and applies via the API.
 - **Backups & DR:** retain GPG/KMS-encrypted offsite backups, restic, 3-2-1, and the quarterly DR drill checklist — but the only stateful thing to back up is **Postgres** (control-plane truth) and the **PKI root**; data-plane nodes are cattle, reprovisioned from IaC in minutes. RPO≈0 (config in Git + Postgres PITR), RTO 15-30 min unchanged.
-

11. Repository layout (Helix-ecosystem aligned)

```

helixvpn/
├─ helix-core/                # umbrella
reconciler, FFI              # Rust: WG control, transport,
├─ ── crates/helix-transport/ # QUIC/MASQUE, Shadowsocks,
UoT, LW0 (shared client+edge)
├─ ── crates/helix-wg/        # WireGuard control +
boringtun fallback

```

```

|   └─ crates/helix-ffi/          # flutter_rust_bridge + UniFFI
surface
└─ helix-edge/                    # Rust: gateway data-plane edge
(uses helix-transport)
└─ helix-go/                      # Go: control plane (identity/
registry/policy/coordinator/pki/telemetry/api)
└─ helix-proto/                  # Protobuf + OpenAPI → generated
Dart/Go/Rust clients
└─ helix-ui/                      # Flutter: design system +
screens + flavors
|   └─ app_access/                # end-user
|   └─ app_connector/            # network-side config
|   └─ app_console/              # admin (web + desktop)
└─ shims/                        # per-platform tunnel providers
|   └─ apple/ android/ windows/ linux/ harmonyos/ aurora/
└─ deploy/                      # Podman quadlets, Terraform,
Grafana-as-code, helixvpctl
└─ helix-agent-base/            # submodule: shared CLAUDE.md/
AGENTS.md/constitution.md

```

This slots into your existing helix-agent-base submodule convention and the Go-backend / cross-platform-frontend / self-hosted-AI-and-infra philosophy already used across HelixGitpx, HelixMemory, HelixSpecifier, etc.

12. Phased roadmap

Phase 0 — Spike (prove the hard parts). WireGuard data path through the gateway edge; helix-transport plain-UDP + one QUIC/MASQUE mode; one client (Linux or Android) on helix-core reaching one connector's LAN. Validates the whole vertical slice.

Phase 1 — MVP self-host. Go control plane (single binary), Postgres+RLS, Redis Streams, WatchNetworkMap, Console (web), Access app on iOS + Android + Linux, Connector daemon. Auto obfuscation ladder with plain/LWO/QUIC. Kill-switch, DNS protection, no-logging. podman quadlet deploy + helixvpctl.

Phase 2 — Parity + reach. Full obfuscation set (Shadowsocks, UoT), DAITA shaping, multi-hop, PQ handshake. Desktop apps (Win/macOS). Policy-as-code + GitOps. HA/multi-region gateways + failover.

Phase 3 — Full platform set. HarmonyOS NEXT and Aurora OS builds (forked Flutter + native tunnel shims). Web WASM MASQUE proxy mode. Multi-tenant Console, audit, optional billing. Security audit + reproducible builds.

13. Key risks & open decisions

Risk / decision	Notes
iOS NE memory ceiling	Hardest constraint; the reason core is Rust. Profile early in Phase 0.
Go vs Rust for the edge QUIC	Spec says Rust (code reuse with clients). If team velocity favors Go, quic-go+masque-go is acceptable — but you lose the single-implementation guarantee. Decide in Phase 0.
HarmonyOS/Aurora effort	UI ports cheaply; tunnel shims are real native work. Don't promise dates until the shims spike.
DAITA correctness	Traffic-analysis defense is subtle; consider adopting the maybenot framework rather than rolling your own.
Overlapping-CIDR UX	4via6 mapping is powerful but confusing to users; invest in Console UX that hides it.
kernel vs userspace WG	Kernel is faster but constrains containerization/permissions; ship both, default to kernel.
Self-host vs SaaS positioning	The same code serves both; decide the licensing (e.g., source-available + commercial) before public release.

14. What to salvage verbatim from the original Home_VPN.md

The original is not wasted — these parts drop straight in:

- The **reverse-tunnel / outbound-dial** core insight (now generalized to all connectors).
- **WireGuard kill-switch** iptables/fwmark rules (now owned by helix-core).
- **Multi-arch Docker buildx** pipeline (retarget to Podman/OCI, same buildx).
- **Prometheus/Grafana/Alertmanager/Blackbox** stack and dashboards (reframed per §9).
- **Terraform** gateway provisioning + floating-IP failover (cloud-init now runs helixvpnctl join).
- **GPG/KMS encrypted backups, restic, 3-2-1, DR drill** checklist (now only Postgres + PKI need backing up).
- **GitOps with sops/git-crypt** (becomes policy-as-code).

- The Hysteria2 QUIC tuning knobs (receive-window ratios, BBR vs Brutal) → useful reference when tuning the quinn MASQUE transport.

What to **drop or demote**: Hysteria2 as the primary QUIC path (replaced by WG-over-MASQUE), the cron-restart health loops (replaced by event-driven reconciliation), and the assumption of a single home LAN (replaced by the multi-connector overlay).

End of refined specification. This document supersedes the architecture framing of Home_VPN.md while preserving its ops content as implementation reference.